Manual – Kernel Data Structures

Код: 4 · Хичээл: F.CSM302 Үйлдлийн систем · БД1

1. Хэрэглэгчийн заавар

1.1 Програмын зорилго

Linux kernel-ийн өгөгдлийн бүтцүүд (**list_head**, queue, stack, bitmap, binary tree)-ийг загварчилж, үйлдэл бүрийг график дээр харуулна. Хэрэглэгч өөрийн оролт, тоо, огноог өгч туршина.

1.2 Дэмжих орчин

UI: орчин үеийн browser (Chrome, Edge, Firefox) – суулгалт шаардлагагүй. CLI: Linux/macOS/WSL, gcc ≥ 9, make. Kernel module: Linux VM + kernel headers.

1.3 Нээх алхам

1) Repository-г татаж задлана. 2) docs/ui/index.html файлыг browser-т нээнэ. 3) Хэрэв CLI хэрэгтэй бол терминалд make ажиллуулна.

1.4 Оролт ба тохиргоо

Бүтэц	Оролт	Хязгаар
list	өдөр, сар, жил	жил 1900–2100, огноо хүчинтэй (оны 2 сар 29-г шалгана)
queue / stack	бүхэл тоо	багтаамж 8 (overflow/underflow мэдээлнэ)
bitmap	n (size), индекс	n = 1..128, индекс 0..n-1
tree	бүхэл тоо	давхардсан утга татгалзана
proc new <нэр> <ажил>	нэр 1–8 тэмдэгт, ажил 1–20 тик	нийт амьд процесс ≤ 8
sched policy fcfs | rr <квант>	квант 1–10	sched tick = 1 нэгж цаг; sched print; sched clear
list find <д> <с> <ж> · queue find <тоо> · stack find <тоо>	хайх үйлдэл	OK … = index/position эсвэл not found
hash put <түлхүүр> <утга> | get | del | size <1..16> | print | clear	1–12 тэмдэгт (үсэг, тоо, _)	нийт ≤ 24 элемент
tree mode bst | avl	зөвхөн хоосон модонд	avl = тэнцвэржсэн мод

1.5 Ажиллуулах, зогсоох, reset

Карт бүрийн товчоор үйлдэл хийнэ, эсвэл «Ахисан түвшин» хэсгийн командын мөрөнд бичнэ, эсвэл script (examples/*.txt) ачаалж «Script ажиллуулах» дарна. «Жишээ үзүүл» товч нь тухайн бүтцийн үйлдлүүдийг автоматаар үзүүлнэ. **Reset** товч бүх бүтцийг цэвэрлэнэ (хуудсыг дахин ачаалбал бас тэр). CLI-г Ctrl+D-ээр зогсооно.

1.6 Дүрслэлийн тэмдэг, өнгө, төлөв

Тэмдэг	Утга
Цагаан хайрцаг, цэнхэр хүрээтэй	node (struct birthday / node)
Цайвар цэнхэрээр дүүргэсэн хайрцаг	сүүлийн үйлдэлд өөрчлөгдсөн/олдсон node
⇄ сум	next ба prev заагч
Цэнхэрээр дүүргэсэн нүд (1)	1 болсон bit; цагаан нүд (0) унтраалттай
Консолын зүүн зураас: цэнхэр / улаан + «Алдаа:»	OK / ERR мессеж

1.7 Жишээ

Жишээ 1 (сурах бичгийн жишээ). «Script» талбарт examples/textbook.txt-ийг ачаалж ажиллуулна. Хүлээгдэх үр дүн: 5 огноо дараалларааа жагсаж, устгасны дараа жагсаалт хоосон болно.

```
OK list add_tail 2/8/1995 (size=1)
OK list add_tail 17/3/1998 (size=2)
OK list add_tail 29/2/2000 (size=3)
OK list add_tail 5/11/2001 (size=4)
OK list add_tail 30/12/2003 (size=5)
LIST size=5: 2/8/1995 -> 17/3/1998 -> 29/2/2000 -> 5/11/2001 -> 30/12/2003
OK list del 17/3/1998 (size=4)
LIST size=4: 2/8/1995 -> 29/2/2000 -> 5/11/2001 -> 30/12/2003
OK list del 2/8/1995 (size=3)
OK list del 29/2/2000 (size=2)
OK list del 5/11/2001 (size=1)
OK list del 30/12/2003 (size=0)
LIST size=0: (empty)
```

Жишээ 3 (процесс ба CPU). «Процесс ба CPU» tab дээр «Жишээ үзүүл» товчийг дарна. Квант 2-той Round Robin-оор A(5 тик), B(3), C(2) процесс ээлжилж ажиллана: C t=5-д, B t=8-д, A t=9-д дуусч, дундаж хүлээлт 5.00 гарна. Мөн FCFS-д шилжүүлж ижил ажлыг дахин туршиж, дундаж хүлээлтийг харьцуул (FCFS-д ердөө 4.33). Командаар: examples/scheduler.txt

```
OK sched policy rr q=2
OK proc new P1 A burst=5 (alive=1)
OK proc new P2 B burst=3 (alive=2)
OK proc new P3 C burst=2 (alive=3)
SCHED policy=rr(q=2) time=0 running=none ready=P1(A:5), P2(B:3), P3(C:2) done=0
TICK t=0 P1(A) remaining=4
TICK t=1 P1(A) remaining=3 preempted
TICK t=2 P2(B) remaining=2
TICK t=3 P2(B) remaining=1 preempted
```

```
TICK t=4 P3(C) remaining=1
TICK t=5 P3(C) remaining=0 finished
TICK t=6 P1(A) remaining=2
TICK t=7 P1(A) remaining=1 preempted
TICK t=8 P2(B) remaining=0 finished
TICK t=9 P1(A) remaining=0 finished
TICK t=10 idle
SCHED policy=rr(q=2) time=11 running=none ready=(empty) done=3
```

Жишээ 4 (хэш хүснэгт ба мод). «Хэш хүснэгт» tab дээр «Жишээ үзүүл» дарвал «ab», «ba» нэг bucket-д орж collision болохыг харна. «Хайлтын мод» tab дээр «1–7 дараалуулж оруул»-ыг энгийн төрлөөр (өндөр 7), дараа нь «Эхнээс нь» → төрөл «Тэнцвэржсэн (AVL)» → «Төрөл сонгох» → «1–7 дараалуулж оруул» (өндөр 3) гэж харьцуул. Командаар: examples/search.txt

```
OK list find 17/3/1998 = index 1
OK list find 1/1/2001 = not found
OK queue find 20 = position 1
OK queue find 99 = not found
OK stack find 1 = position 2
OK stack find 7 = not found
OK hash size 4
OK hash put alice -> 1234 (bucket=2, size=1)
OK hash put bob -> 5678 (bucket=3, size=2)
OK hash put carol -> 9 (bucket=1, size=3)
OK hash put dave -> 11 (bucket=0, size=4)
OK hash put ab -> 7 (bucket=3, size=5, collision)
OK hash put ba -> 8 (bucket=3, size=6, collision)
HASH buckets=4 size=6 collisions=2: b0=[dave:11] b1=[carol:9] b2=[alice:1234] b3=[bob:5678 -> ab:7 -> ba:8]
OK hash get ba = 8 (bucket=3, steps=3)
OK hash get bob = 5678 (bucket=3, steps=1)
OK hash get zed = not found (bucket=3)
OK hash put bob = 0 (updated, bucket=3)
OK hash del alice (size=5)
ERR hash: key not found
HASH buckets=4 size=5 collisions=2: b0=[dave:11] b1=[carol:9] b3=[bob:0 -> ab:7 -> ba:8]
ERR hash: key and value must be 1..12 letters, digits or _
TREE size=5 height=5 inorder: 1, 2, 3, 4, 5
OK tree find 5 = yes
TREE(avl) size=7 height=3 inorder: 1, 2, 3, 4, 5, 6, 7
OK tree find 5 = yes
TREE(avl) size=6 height=3 inorder: 1, 2, 3, 5, 6, 7
```

Жишээ 2 (өөрийн оролт). bitmap size 16, bitmap set 3, bitmap set 9 гэж бичвэл 16 нүдний 3, 9-р нүд ногоон болно; bitmap print → 0001000001000000 (set=2).

1.8 Түгээмэл алдаа ба шийдэл

Мессеж	Шалтгаан	Шийдэл
ERR list: day out of range for month	Байхгүй огноо (31/4, 29/2/1900)	Зөв огноо оруулах
ERR queue: overflow (limit 8)	8-аас олон push	pop хийх
ERR queue: underflow (empty)	Хоосон дээр pop	Эхлээд push
ERR bitmap: index N out of range	Индекс хэтэрсэн	0..n-1 дотор оруулах
ERR tree: duplicate	Ижил утга	Өөр утга
ERR proc: name must be ...	Нэр урт/тусгай тэмдэгттэй	Ж: A, job1
ERR proc: too many processes	8-аас олон амьд процесс	Эхлээд зарим нь дуустал нь тик хий
ERR hash: key and value must be …	Түлхүүр/утга 12-оос урт эсвэл «-» гэх мэт тэмдэгттэй	Ж: alice, key_1
ERR tree: mode can only be changed …	Мод хоосон биш	Эхлээд «Эхнээс нь» / бүх тоог хас

2. Operational заавар

2.1 Repository clone

```
git clone <github-холбоос> kernel-data-structures
cd kernel-data-structures
```

2.2 Орчин

Ubuntu 24.04 (туршсан), gcc, GNU make, node ≥ 18 (зөвхөн UI==C тестэд), python3 (зөвхөн screenshot/PDF-д). Dependency нэмж суулгах шаардлагагүй.

2.3 Build ба Run

```
make
./kds examples/textbook.txt
echo "queue push 5" | ./kds     # stdin-ээс
```

2.4 Үндсэн тестийн өгөгдөл

```
make test
# PASS textbook / all_structures / errors / scheduler / scheduler_full / search / stress
python3 tests/fuzz_compare.py 2 5000   # санамсаргүй 5000 команд: C == UI (тэгш seed-д AVL)
python3 tests/avl_check.py             # AVL-ийн өндөр ба агуулгын шалгалт
sh tests/test_ui_matches_c.sh   # UI-ийн хөдөлгүүр C-тэй ижил гаралт өгч байгааг шалгана
```

2.5 Нэмэлт boundary/error test

examples/errors.txt: буруу огноо, хоосон бүтцээс pop, size 0 ба 129, индекс −1, давхардсан tree утга. Бүх мөр ойлгомжтой ERR мессеж өгөх ёстой.

2.6 Kernel module (Linux VM)

```
cd src/kernel
make -C /lib/modules/$(uname -r)/build M=$PWD modules
sudo insmod kds_module.ko && sudo dmesg | tail -20
sudo rmmod kds_module && sudo dmesg | tail -8
```

Баталгаажуулалт: модуль Ubuntu 24.04, gcc-13, linux-headers-7.0.0-38-generic-ээр алдаа, warning-гүй compile хийгдэж kds_module.ko үүссэн (docs/kernel_build_log.txt). Өөрийн машин (kernel 6.11.0-17-generic) дээр insmod/rmmod амжилттай хийгдсэн.

```
# Secure Boot идэвхтэй үед (insmod: Key was rejected by service) – нэг удаа:
mkdir -p ~/mok && cd ~/mok
```

```
openssl req -new -x509 -newkey rsa:2048 -keyout MOK.priv -outform DER -out MOK.der -nodes
-days 3650 -subj "/CN=KDS module signing/"
sudo mokutil --import MOK.der      # дараа нь reboot, MOK дэлгэц дээр Enroll MOK
# build бүрийн дараа:
sudo /usr/src/linux-headers-$(uname -r)/scripts/sign-file sha256 ~/mok/MOK.priv
~/mok/MOK.der kds_module.ko
```

Хүлээгдэх dmesg: 5 «kds: birthday d/m/y», fifo-ooc 10, 20, 30, «bitmap weight=2, bit15=1», rmmod үед 5 «removing» мөр.